

Topology-Aware Neural Architecture Fingerprinting via Inference Timing Leakage

Shlok Pandey

Manipal University Jaipur

Email: shlokpandey8219@gmail.com

Abstract—Latency-based side-channel leakage has emerged as a major security concern in modern machine-learning-as-a-service (MLaaS) deployments, where models are exposed through black-box query-response interfaces. Prior work on neural network reverse engineering has largely focused on hardware traces, electromagnetic leakage, cache behavior, or output-based model extraction. In contrast, this work investigates whether inference latency alone can be exploited to fingerprint neural network architectures under constrained black-box access settings. Statistical timing descriptors including mean latency, standard deviation, tail latency, skewness, kurtosis, and temporal autocorrelation are extracted from repeated inference observations and used to train lightweight architecture classifiers. Experimental results demonstrate that architectures including VGG, ResNet, MobileNetV2, and InceptionV3 produce statistically distinguishable timing signatures, enabling effective identification using latency information alone. Additional experiments analyze query-count sensitivity and robustness under noisy timing conditions. The findings further demonstrate that hardware environment heterogeneity substantially reduces fingerprinting transferability, a result with direct implications for MLaaS defense design.

I. INTRODUCTION

Most cloud platforms and machine learning services provide hosted models through API-based interfaces. These services permit users to query deployed models through query-response interfaces. These systems generally run in black-box deployment environments where users have limited visibility into the underlying architecture or implementation characteristics. In settings like these, systems may unintentionally leak information through side channels such as inference latency [1]–[3].

Recent work has shown that model behavior, query patterns, hardware traces, and implementation-level signals can expose architectural details about neural networks [2]–[5]. However, many existing approaches rely on hardware-level traces, electromagnetic leakage, or richer side-channel information beyond simple request-response timing. These risks are especially relevant as machine-learning-as-a-service platforms increasingly expose high-value models through black-box interfaces. This raises an important question: whether inference latency alone can reveal architecture-level information under constrained black-box access conditions.

In this work we specifically investigate inference latency as a possible side-channel for neural network architecture fingerprinting. We methodically investigate whether the response time of neural network inference can reveal structural information about the underlying model, which may subsequently assist surrogate model construction and model

stealing attacks [4], [6]. We hypothesize that differences in the computational graph topology across architectures like VGG, ResNet, MobileNetV2 and Inception can leak distinct latency signatures that can be exploited for model fingerprinting.

A. Research Gap and Contributions

Existing research on reverse engineering of neural networks and model extraction have relied upon hardware trace data, electromagnetic emission data, cache data or output based extraction. In comparison to this body of research, there is relatively little existing research examining if only inference latency could provide a means of deriving architecture level information in situations where the user has very restricted black box access. The objective of this work is to investigate if computational graph structure produces timing signatures which are statistically distinguishable from one another and therefore exploitable as a basis for identifying neural network architectures solely through request response timing observation.

The primary contributions to this work are:

- We present Neural Architecture Fingerprinting (NAF) as a latency-only Black-Box Side Channel Problem.
- We determine if the Computational Graph Topology is sufficient to induce Statistically Distinguishable Timing Signatures in VGG, ResNet, MobileNetV2 and InceptionV3 architectures.
- We test both Statistical Latency Descriptors (Mean Latency, Variance, Skewness, Kurtosis), and Temporal Latency Descriptors (Autocorrelation).
- We assess how Query Count and Noisy Timing Conditions impact NAF Performance.
- We will also address Security Implications, Limitations, and Countermeasures to prevent Timing-Based Architecture Leakage.

II. BACKGROUND

A. Directed Acyclic Graphs (DAGs)

Modern neural network architectures can be modeled as DAGs, where the execution flow propagates through interconnected computational layers. Sequential models such as VGG primarily show a linear computational graph, whereas architectures such as ResNet and Inception exhibit branching, merging, and parallel execution as they operate on the principles of skip connections and multi-scale feature extraction [7], [8]. These core structural differences alter the execution traits of the



Fig. 1. Simplified VGG-style sequential computation graph.

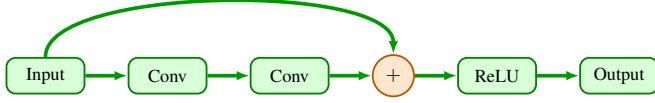


Fig. 2. Simplified ResNet residual block with skip connection and merge operation.

model, including inference latency, synchronization overhead, and computational variability. Consequently, DAG topology becomes an important factor in understanding latency-based side-channel behavior in neural network systems.

B. Neural Network Architectures

The evaluated architectures are representative of distinct computational graph topologies. VGG follows a strictly sequential execution structure. ResNet introduces skip connection which provides a path to bypass certain layers or operations with the help of residual learning [7]. MobileNetV2 uses an approach called Depthwise Separable Convolutions that allow it to be able to operate in real-time by optimizing its use of memory [9]. Also, InceptionV3 utilizes multiple branches of convolutional layer [8] as one way to create parallelism during the inference process, which produce a branch-and-merge computation graph. These topology differences influence execution flow, synchronization behavior, and inference latency variability.

C. Side-Channel Attacks

Side-channel attacks exploit indirect execution signals such as timing behavior, hardware traces, cache activity, or memory-access patterns to infer information about a target system [1], [3]. In machine learning systems, observable execution behavior may unintentionally leak architectural or implementation-level information about deployed neural networks [2], [5]. Inference latency represents one of the most accessible side channels in black-box ML deployments because response timing can often be measured without privileged hardware access.

Different neural network architectures exhibit distinct execution behavior due to variations in computational graph topology, synchronization behavior, branching patterns, and computational complexity. These topology differences may therefore induce statistically distinguishable timing signatures across deployed neural network architectures.

Recent studies on neural-network reverse engineering and model extraction have shown that architectural hints, hardware traces, and query observations can leak information about deployed neural networks [2], [3], [5], [10]. As a result, different architectures may produce distinct latency distributions during inference. In black-box deployment settings, an adversary may repeatedly query the model and analyze these timing variations to infer structural information about the underlying architecture.

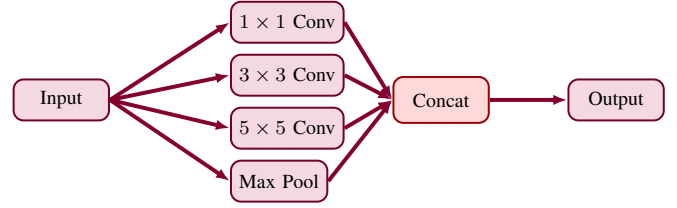


Fig. 3. Simplified Inception-style multi-branch module with concatenation merge.

D. Comparison with Prior Work

Recent studies have showcased that neural network architectures and implementation properties might leak through various side channels including electromagnetic emissions, hardware traces, cache behavior, and execution timing. However, many existing approaches depend on privileged hardware access or specialized measurement environments. In contrast, this work investigates whether inference latency alone can reveal architecture-level information under constrained black-box access scenarios without requiring direct hardware trace access.

TABLE I
COMPARISON WITH PRIOR NEURAL NETWORK SIDE-CHANNEL STUDIES

Work	Side Channel	Target	HW Dep.
CSI NN	EM Leakage	Architecture	Yes
DeepSniffer	Hardware Traces	DNN	Yes
PRADA	Query Behavior	Model Extraction	No
This Work	Inference Latency	Architecture	No

As shown in Table I, prior neural network side-channel studies frequently rely on hardware-level leakage signals or richer execution traces. In comparison, the proposed methodology focuses exclusively on inference latency observations obtained through a standard black-box query-response interface. This enables evaluation of architecture fingerprinting feasibility under significantly more constrained attacker capabilities.

E. Black-Box Setting

The experiment presumes a black-box deployment environment in which the attacker has no visibility into the internal architecture, parameters, data used in training, or implementation insights of the deployed neural network model. The target is accessible solely through a request-response interface. Despite complete restriction from internal access, the adversary can repeatedly query the model and observe inference response times. These timing records are further analyzed in order to infer structural characteristics of the underlying model.

III. THREAT MODEL

The threat model assumes an adversary interacting with a remotely hosted neural network through a black-box API interface. The attacker does not observe the architecture,

model weights, training data, hardware configuration, gradients, logits, intermediate activations, or implementation details of the deployed model. However, the adversary can repeatedly query the target system and record inference response times. These timing observations are analyzed to infer architecture-level information about the underlying neural network. In summary, the adversary has black-box API access, observes only inference latency, can issue repeated queries, and aims to perform architecture fingerprinting without access to model internals or hardware traces.

IV. METHODOLOGY

A. Model Selection

Four convolutional neural network (CNN) architectures representing distinct computational graph topologies were selected for evaluation: ResNet-18, ResNet-50, VGG-16, InceptionV3, and MobileNetV2. They were selected because they exhibit distinct execution patterns and computational graph structures that may influence inference latency behavior. They were loaded from `torchvision.models` [7], [8], [11] and were selected because they represent significantly different computation graph topologies. VGG primarily follows a sequential execution structure, whereas ResNet introduces skip connections and residual blocks that create branch-and-merge computation paths. MobileNetV2 utilizes depthwise separable convolutions optimized for lightweight inference [9]. InceptionV3 employs parallel branches and concatenation, resulting in highly parallel execution paths.

These architectural differences make the selected models suitable for evaluating whether computational graph structure influences inference latency and timing behavior.

B. Latency Measurement

Inference latency measurements were conducted on an AMD Ryzen 7 5800H CPU with 16GB RAM running Ubuntu 22.04 and PyTorch 2.2. Additional experiments were performed using CUDA-enabled GPU inference in order to evaluate latency fingerprinting behavior across heterogeneous hardware environments. Baseline experiments were conducted on CPU execution with batching disabled, while additional heterogeneous hardware experiments were conducted using CUDA-enabled GPU inference. All models received identical input tensors of size $(1, 3, 224, 224)$ to ensure consistent evaluation conditions. Each model was switched to evaluation mode using `model.eval()` in order to disable training-specific behavior during inference. In addition, `torch.no_grad()` was utilized to disable gradient computation and reduce unnecessary overhead during latency measurement. A warm-up inference pass was executed prior to timing measurements to reduce initialization overhead.

Each architecture executed 100 inference cycles, with individual inference times recorded using `time.perf_counter()`. The duration of every forward pass was measured in milliseconds, producing a latency distribution for each architecture. Statistical and temporal timing descriptors including mean latency, standard deviation,



Fig. 4. Latency measurement and feature extraction pipeline utilized during inference experiments.

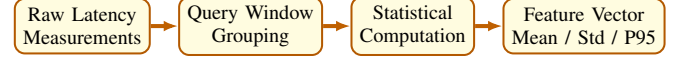


Fig. 5. Statistical feature extraction pipeline used for latency fingerprint generation.

minimum latency, maximum latency, P95 latency, skewness, kurtosis, and lag-1 autocorrelation were subsequently extracted from the latency distributions. Figure 4 illustrates the latency measurement and feature extraction workflow.

C. Feature Extraction

Raw latency measurements were transformed into statistical and temporal timing descriptors for architecture fingerprint generation. For a latency sequence $L = \{l_1, l_2, \dots, l_n\}$, features including mean latency, standard deviation, minimum latency, maximum latency, P95 latency, skewness, kurtosis, and lag-1 autocorrelation were extracted from the recorded inference timings. These descriptors were used to generate timing fingerprints for downstream architecture classification tasks.

Figure 5 illustrates the statistical feature extraction pipeline utilized throughout the experiments.

D. Classification Approach

Latency-derived feature vectors were used to train lightweight architecture classification models. Each feature vector represented the statistical timing characteristics of a specific neural network architecture including ResNet-18, ResNet-50, VGG-16, MobileNetV2 and InceptionV3.

Model evaluation was performed using 5-fold cross-validation with an 80/20 train-test split across folds. Prior to classification, feature normalization was applied in order to reduce scale variation across different statistical descriptors. K-Nearest Neighbors (KNN) and Logistic Regression classifiers were subsequently trained and evaluated on the extracted latency features. Performance was evaluated using mean classification accuracy, macro-F1 score, confusion matrices, and per-class precision-recall metrics across validation folds. KNN was evaluated using $k = 5$, while Logistic Regression was configured with standard L2 regularization.

Classification accuracy, confusion matrices, and classification reports were utilized to evaluate the effectiveness of latency-based architecture fingerprinting. Figure 6 illustrates the classification workflow implemented during the experiments.

V. RESULTS

A. Latency Measurements

Table II summarizes the observed latency characteristics across all evaluated architectures.

Table II shows the latency characteristics observed across the evaluated neural network architectures. MobileNetV2 exhibited the lowest average inference latency and the lowest



Fig. 6. Latency-based architecture classification workflow utilized during experiments.

TABLE II
LATENCY STATISTICS FOR DIFFERENT ARCHITECTURES

Model	Mean (ms)	Std Dev (ms)	P95 (ms)
ResNet-18	52.91	22.83	90.34
ResNet-50	135.66	36.01	207.34
VGG-16	239.66	32.38	300.04
InceptionV3	74.27	5.72	82.03
MobileNetV2	30.12	2.01	33.77

Inference Latency Distributions per Model

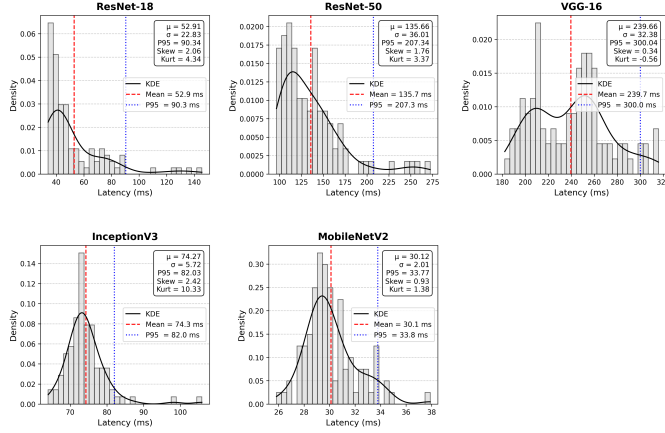


Fig. 7. Inference latency distributions observed across evaluated CNN architectures. Red dashed lines indicate mean latency, while blue dashed lines indicate P95 latency.

TABLE III
CROSS-VALIDATION CLASSIFICATION PERFORMANCE

Classifier	Mean Acc. (%)	Std Dev (%)	Macro-F1
KNN	94.7	1.8	0.946
Logistic Regression	95.0	2.1	0.949

latency variance, whereas VGG-16 exhibited the highest average latency and significantly larger tail latency behavior. ResNet-50 also demonstrated relatively high latency variance compared to lighter architectures such as ResNet-18 and MobileNetV2.

Figure 7 illustrates the latency distributions generated during repeated inference execution. Architectures with deeper or more computationally intensive computation graphs demonstrated broader latency distributions and larger P95 latency values. In contrast, lightweight architectures exhibited more concentrated latency behavior with reduced timing dispersion.

B. Classification Accuracy

Table III presents the architecture identification performance achieved using latency-derived features.

Table III summarizes the classification performance achieved using latency-derived statistical timing descriptors.

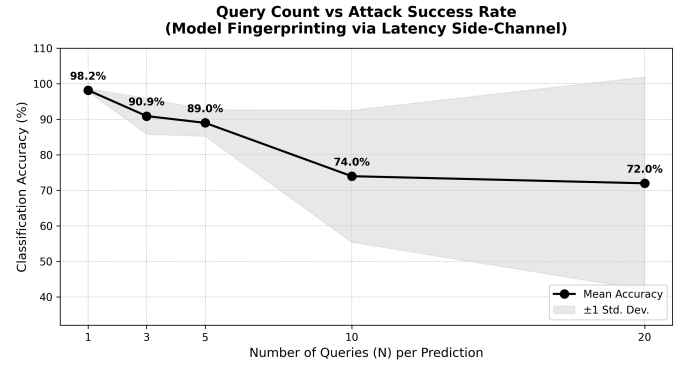


Fig. 8. Classification accuracy under varying query budgets.

Both KNN and Logistic Regression achieved high classification accuracy across the evaluated architectures, with Logistic Regression slightly outperforming KNN in mean cross-validation accuracy and macro-F1 score. These results indicate that statistical latency features preserve architecture-specific execution characteristics that remain distinguishable under black-box inference conditions.

The classification results demonstrate that statistical timing descriptors extracted from inference latency preserve architecture-specific execution characteristics. These findings suggest that inference latency can function as a practical side-channel for neural network architecture fingerprinting under constrained black-box access conditions.

C. Effect of Query Count

Figure 8 summarizes classification performance under varying query budgets. Under controlled CPU execution, single-query observations achieved 98.2% accuracy, likely reflecting strong mean-latency separation between architectures in an uncontended environment. This sensitivity degrades substantially under realistic multi-hardware conditions, suggesting that query-count alone is not the primary bottleneck for practical fingerprinting — execution environment consistency is. Classification performance gradually decreased as larger independent query windows were utilized. This likely reflects that larger query windows average out localized latency spikes that contribute to inter-architecture separability.

D. Noise Robustness

As shown in Figure 9, classification performance eventually decreased as rising levels of Gaussian timing noise appeared into the latency readings. Both KNN and Logistic Regression remained stable under low-noise scenarios, although KNN demonstrated larger degradation in performance at higher noise magnitude. Logistic Regression exhibited comparatively improved robustness under noisy timing conditions. Despite the injected timing perturbations, both KNN and Logistic Regression retained meaningful classification capabilities, suggesting that latency-derived statistical features preserve partially distinguishable architectural timing characteristics even in noisy inference environments.

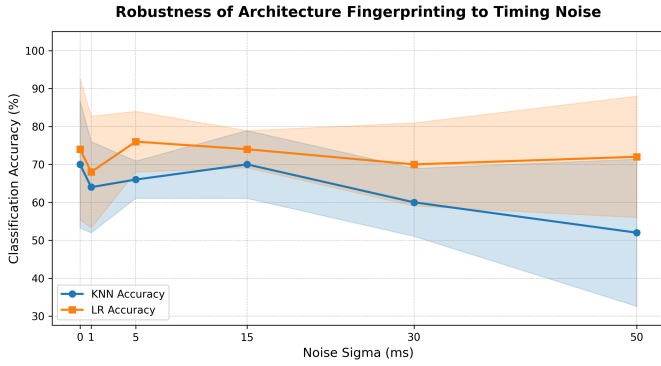


Fig. 9. Noise robustness evaluation under injected Gaussian timing perturbations.

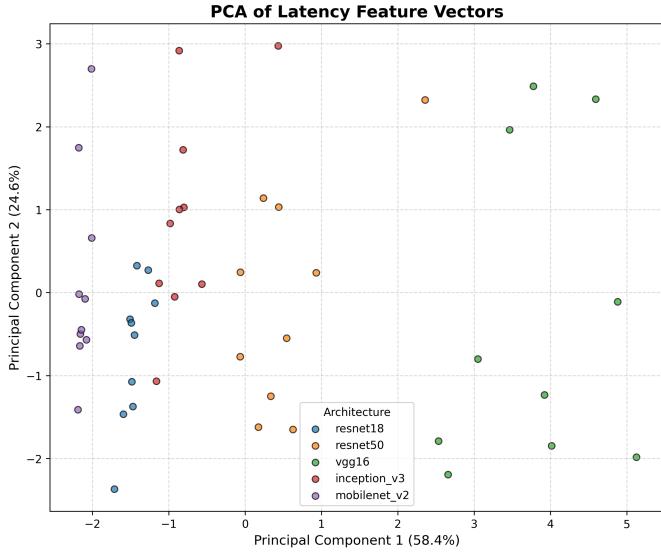


Fig. 10. PCA visualization of latency-derived feature vectors.

E. Feature-Space Visualization

Figure 10 illustrates PCA projections of the extracted latency feature vectors. Distinct clustering behavior can be observed across multiple architectures, indicating that the proposed statistical timing descriptors preserve separable execution characteristics.

F. Heterogeneous Hardware Evaluation

To evaluate the robustness of latency-based architecture fingerprinting across different execution environments, additional experiments were conducted using both CPU and GPU inference execution. Timing measurements were collected independently for CPU-only, GPU-only, and mixed hardware datasets.

Figure 11 illustrates the latency distributions observed across CPU and GPU execution environments. GPU inference generally produced lower average latency and reduced variance compared to CPU execution due to parallelized execution behavior and hardware acceleration effects.

Table IV summarizes the classification performance achieved under CPU-only, GPU-only, and mixed hardware

TABLE IV
CLASSIFICATION ACCURACY ACROSS HARDWARE ENVIRONMENTS

Dataset	Classifier	Accuracy (%)	Macro-F1
CPU	KNN	66.0	0.65
CPU	Logistic Regression	70.0	0.69
GPU	KNN	56.0	0.55
GPU	Logistic Regression	68.0	0.67
Mixed	KNN	41.0	0.40
Mixed	Logistic Regression	43.0	0.42

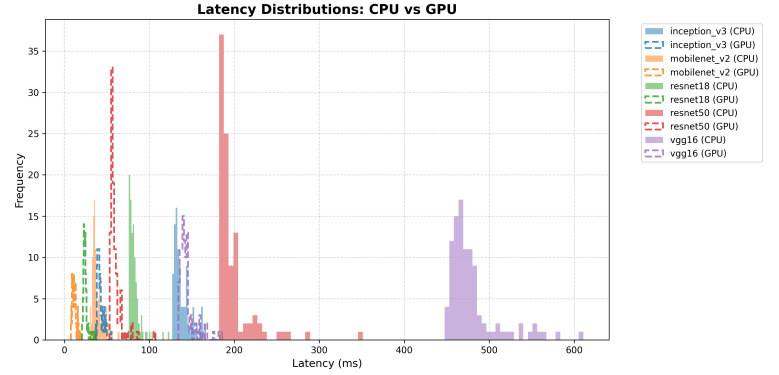


Fig. 11. Latency distributions observed across CPU and GPU inference environments for the evaluated architectures. GPU execution generally exhibited lower latency and reduced timing variance compared to CPU execution.

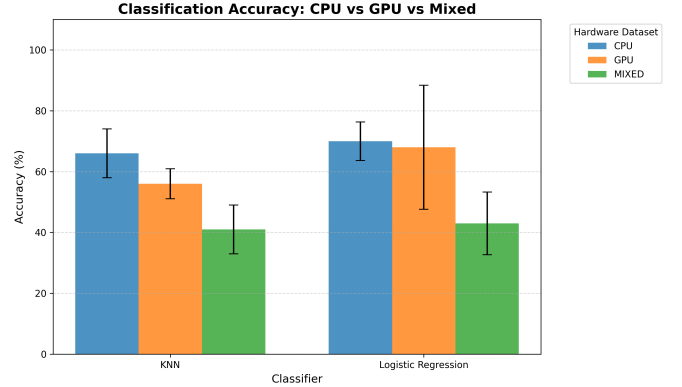


Fig. 12. Architecture classification accuracy across CPU-only, GPU-only, and mixed hardware datasets using KNN and Logistic Regression classifiers.

conditions. Classification accuracy decreased under mixed hardware conditions, indicating that hardware-specific timing behavior introduces additional variability that affects architecture separability.

Figure 13 further demonstrates that latency-derived feature vectors exhibit observable distribution shifts across heterogeneous hardware environments. Despite these variations, architecture-dependent timing structure remained partially distinguishable across execution platforms.

The heterogeneous hardware experiments showcase that latency-based neural network fingerprinting remains somewhat partially effective across different execution environments, although hardware-specific timing behavior introduces additional variability that reduces classification separability under

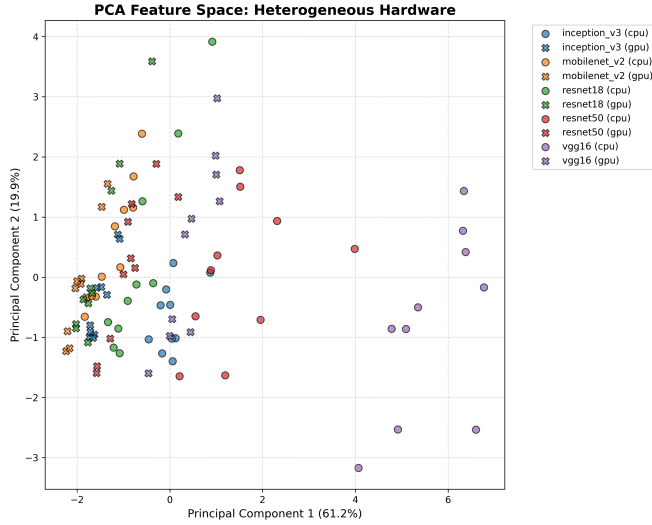


Fig. 13. PCA visualization of latency-derived feature vectors across heterogeneous hardware environments. Marker shape indicates hardware type, while color represents neural network architecture.

mixed deployment conditions. The observed distribution shifts between CPU and GPU inference indicate that execution hardware itself contributes measurable timing characteristics that influence side-channel leakage behavior.

VI. DISCUSSION AND CONCLUSION

The observed accuracy difference between controlled CPU-only evaluation (95%) and mixed hardware conditions (43%) represents a fundamental property of timing-based side channels: latency fingerprints are partially hardware-environment-specific rather than purely architecture-specific. This result highlights that hardware heterogeneity and inference routing variability may function as partial mitigations against timing-based architecture leakage in MLaaS deployment settings. The experimental results demonstrate that inference latency preserves architecture-dependent timing characteristics that remain exploitable under black-box conditions. Distinct latency distributions were observed across all evaluated architectures, and meaningful classification performance was retained even under injected timing noise. The evaluation was conducted under relatively controlled execution conditions, however, and may not fully capture real-world deployment effects such as cloud scheduling behavior, multi-tenant inference workloads, and network-induced timing variability. Several countermeasures against timing-based architecture leakage are identified. Latency normalization eliminates timing variance by padding inference responses to a fixed response time. Random latency injection introduces artificial noise to obscure architecture-specific timing signatures. Hardware routing randomization reduces cross-query timing consistency, as suggested by the degraded classification performance observed under mixed hardware conditions. Rate limiting and query anomaly detection can additionally constrain the number of timing observations available to an adversary.

VII. FUTURE WORK

Future work may extend the proposed methodology to larger-scale heterogeneous deployment environments including cloud inference systems, edge accelerators, and mobile devices in order to evaluate the consistency of latency-based fingerprinting under realistic multi-platform execution conditions. Additional investigation may also include larger neural network families such as transformer architectures, hybrid models, and quantized networks, along with more advanced temporal and frequency-domain timing descriptors. Evaluating the effects of realistic deployment conditions including network jitter, concurrent workloads, caching behavior, and asynchronous execution pipelines may further improve understanding of timing leakage in practical ML deployment environments. Future research may additionally explore defensive mechanisms designed to mitigate timing-based information leakage during neural network inference.

REFERENCES

- [1] P. Kocher, "Timing Attacks on Implementations of Diffie-Hellman, RSA, DSS, and Other Systems," in *Proceedings of the Annual International Cryptology Conference (CRYPTO)*, 1996.
- [2] L. Batina, S. Bhasin, D. Jap, and S. Picek, "CSI NN: Reverse Engineering of Neural Network Architectures Through Electromagnetic Side Channel," in *Proceedings of the USENIX Security Symposium*, 2019.
- [3] J. Wei et al., "Machine Learning Model Extraction Through Hardware Side Channels," in *Proceedings of the IEEE International Symposium on Hardware Oriented Security and Trust (HOST)*, 2020.
- [4] M. Juuti, S. Szyller, S. Marchal, and N. Asokan, "PRADA: Protecting Against DNN Model Stealing Attacks," in *Proceedings of the IEEE European Symposium on Security and Privacy (EuroS&P)*, 2019.
- [5] X. Hu, L. Liang, S. Li, L. Deng, P. Zuo, Y. Ji, X. Xie, Y. Ding, C. Liu, T. Sherwood, and Y. Xie, "DeepSniffer: A DNN Model Extraction Framework Based on Learning Architectural Hints," in *Proceedings of the ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2020.
- [6] T. Orekondy, B. Schiele, and M. Fritz, "Knockoff Nets: Stealing Functionality of Black-Box Models," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.
- [7] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.
- [8] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna, "Rethinking the Inception Architecture for Computer Vision," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.
- [9] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted Residuals and Linear Bottlenecks," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018.
- [10] M. Hua, Y. Wang, S. Wang, and H. Qian, "Reverse Engineering Convolutional Neural Networks Through Side-Channel Information Leaks," in *Proceedings of the Design Automation Conference (DAC)*, 2018.
- [11] K. Simonyan and A. Zisserman, "Very Deep Convolutional Networks for Large-Scale Image Recognition," in *International Conference on Learning Representations (ICLR)*, 2015.